



Projektarbeit

Von Capabilities zu einem Messaging System

vorgelegt von

Marek Westermann

aus Düsseldorf

Abteilung Betriebssysteme
Prof. Dr. Michael Schöttner
Heinrich-Heine-Universität Düsseldorf

7.September 2026

Erstgutachter: Prof. Dr. Michael Schöttner
Betreuer: Prof. Dr. Michael Schöttner

Inhaltsverzeichnis

1	Einleitung	1
1.1	Checkliste	2
2	Optimierung, Stabilisierung und Integration des Capability-Subsystems	3
2.1	Motivation und Ausgangslage	3
2.2	Synchronisationsoptimierung: Umstellung von Mutex auf Reader-Writer-Locks	3
2.2.1	Problemstellung und Flaschenhals	3
2.2.2	Implementierte Lösung	4
2.2.3	Anpassung der Aufrufstellen und Resultat	4
2.3	Behebung von Kernel-Panics im Fehlerpfad des Dispatchers	4
2.3.1	Fehleranalyse	4
2.3.2	Korrektur und Stabilität	5
2.4	Rebase und Synchronisation mit dem aktuellen Kernel-Stand	5
2.4.1	Konsolidierung der ABI und Datenstrukturen (<code>shared_types.rs</code>)	5
2.4.2	Namensraum-Restriktionen und Sicherheitsaspekte (<code>naming/lib.rs</code> , <code>sys_naming</code>)	5
2.4.3	Überarbeitung des Syscall-Dispatchers und Low-Level-Register-Management	6
2.4.4	Testumgebung und Verifikation (<code>api.rs</code> , <code>test/pipe-rs</code>)	6
3	Gliederung der Arbeit	8
4	Struktur	9
4.1	Mein Abschnitt	10
4.1.1	Unterabschnitt	10
4.1.2	Noch ein Unterabschnitt	10
4.1.3	Wieder eine Subsection	10
4.2	Und eine einfache Section	10
4.3	Zeilenumbrüche und Absätze	10
4.4	Seitenumbrüche	10
5	Organisation	11
6	Text	12
7	Zitieren	13
8	Auflistungen	14
8.1	Enumeration	14

8.2	Bullet Points	14
8.3	Eigene Symbole	14
9	Bilder und Grafiken	15
10	Tabellen	17
10.1	Einfache Tabelle	17
10.2	Komplexere Tabelle: Multicolumn und Multirow	17
11	Matheumgebung	18
11.1	Superscript und Subscript	18
11.2	Zentrierte Formeln	18
11.3	Symbole	18
12	Code	19
12.1	Pseudocode	19
12.2	Codeschnipsel	20
12.3	Terminalausgabe	20
	Literatur	21
	Abbildungsverzeichnis	22
	Tabellenverzeichnis	23
	Algorithmenverzeichnis	24

Kapitel 1

Einleitung

Nachfolgend einige Punkte, die es zu beachten gilt:

1. Dieses Template dient als Vorlage, damit diverse Anforderungen wie allgemeines Layout der Arbeit, Formatierung etc. einheitlich sind. Weiterhin bietet es einen etwas leichteren Einstieg in LaTeX und sollte somit nicht zu viel Zeit in Anspruch nehmen, damit die eigentliche Arbeit nicht darunter leidet.
2. Einzelne Kapitel dieser Vorlage geben einige grundlegende Konstrukte vor, die zum größten Teil per Copy/Paste übernommen werden können. Dennoch ist es unvermeidbar, sich mit LaTeX (bis zu einem gewissen Maß) zu beschäftigen. Mit Google kann man die meisten Probleme sehr leicht lösen und bekommt oft direkt den passenden Code geliefert.

1.1 Checkliste

Folgende Punkte sollten vor der Abgabe der Arbeit unbedingt noch einmal geprüft werden (keine Garantie auf Vollständigkeit!).

- ☐ Das Dokument ist auf doppelseitigen Druck ausgelegt, d.h. unbedingt doppelseitig drucken lassen.
- ☐ Vor dem Hochladen **alles** nochmal kontrollieren.
- ☐ Sind alle Kapitel vorhanden?
- ☐ Ist das Inhaltsverzeichnis vollständig?
- ☐ Sind alle Bestandteile der Arbeit vorhanden (Titelblatt, Inhaltsverzeichnis, Anhang, Erklärung, etc.)
- ☐ Sind leere Abschnitte entfernt (z.B. keine Algorithmen im Algorithmusverzeichnis)?
- ☐ Sind alle Todos aus dem Dokument entfernt und erledigt?
- ☐ Steht das richtige Datum auf der Arbeit?
- ☐ Das Thema muss **exakt** so formuliert sein, wie bei der Anmeldung der Arbeit.
- ☐ Die Arbeit ist digital als PDF im Studierendenportal einzureichen.
- ☐ Jegliche digitalen Dokumente, Quellcode, Programme, Messergebnisse und Anleitungen für den Aufbau von Testumgebungen sowie der LaTeX-Quelltext zur Arbeit selbst müssen zum Zeitpunkt der Abgabe im GitLab Repository vorhanden sein.

Kapitel 2

Optimierung, Stabilisierung und Integration des Capability-Subsystems

2.1 Motivation und Ausgangslage

Moderne Mikrokern- und Betriebssystemarchitekturen setzen zur Reduzierung der Angriffsfläche konsequent auf das Prinzip der minimalen Rechtevergabe (*Principle of Least Privilege*). Ein zentraler Baustein hierfür sind Capabilities: kryptografisch oder architektonisch unschmiedbare Token, die eine Ressource referenzieren und die darauf zulässigen Operationen fest an die Identität des Tokens binden.

Die vorliegende Arbeit baut direkt auf der Masterarbeit von Julian Schnock auf, deren Entwicklungsstand aus September 2025 datiert. In dieser Vorarbeit wurde ein grundlegendes Capability-System für das Betriebssystem implementiert, welches auf typisierten Capabilities sowie task-lokalen Capability Spaces (CSpaces) basiert. Obwohl diese Basisarchitektur die grundlegende Machbarkeit nachwies, wies der Stand von September 2025 signifikante Schwachstellen hinsichtlich Nebenläufigkeit und Fehlerbehandlung auf. Zudem existierte eine erhebliche Versionsdiskrepanz zum weiterentwickelten Hauptzweig des Kernels.

Das Ziel dieses einleitenden Kapitels ist die Beseitigung bestehender Synchronisations- und Stabilitätsprobleme sowie die Zusammenführung (Merge) des Capability-Zweigs mit dem aktuellen Kernel-Stand als Fundament für die Integration des Messaging-Systems und von Shared Memory.

2.2 Synchronisationsoptimierung: Umstellung von Mutex auf Reader-Writer-Locks

2.2.1 Problemstellung und Flaschenhals

Im ursprünglichen Entwurf wurden gekapselte Ressourcen innerhalb einer `Capability<T>` mittels eines gegenseitigen Ausschlusses (`obj: Option<Arc<Mutex<T>>>`) synchronisiert. Ebenso erfolgte der Zugriff auf den CSpace ausschließlich über eine strikt exklusive `.invoke()`-Methode.

Bei der praktischen Ausführung von Systemaufrufen im Dispatcher dominieren jedoch Lesezugriffe (z. B. das Auslesen von Zugriffsrechten, Deskriptorprüfungen oder das Abfra-

gen von Metadaten). Durch den Einsatz von Mutices sperrten sich nebenläufige Threads gegenseitig aus, selbst wenn parallel lediglich lesend auf dieselben Datenstrukturen zugegriffen wurde (*Lock Contention*). Dies verhinderte die Skalierung auf Mehrkernsystemen und stellte einen massiven Durchsatzflaschenhals dar.

2.2.2 Implementierte Lösung

Um paralleles Lesen zu ermöglichen, wurde die interne Synchronisation von `Mutex<T>` auf `RwLock<T>` umgestellt. Dies erforderte eine semantische Auftrennung der Zugriffsschnittstelle in zwei Pfade:

Nebenläufiger Lesezugriff (`invoke`):

```
1 pub fn invoke(&self) -> Option<RwLockReadGuard<'_, T>> {  
2     self.obj.as_ref()?.try_read()  
3 }
```

Über `try_read()` erhalten beliebig viele Threads simultanen Lesezugriff auf die Ressource, ohne einander zu blockieren.

Exklusiver Schreibzugriff (`invoke_mut`):

```
1 pub fn invoke_mut(&self) -> Option<RwLockWriteGuard<'_, T>> {  
2     self.obj.as_ref()?.try_write()  
3 }
```

Über `try_write()` wird ein exklusiver Lock angefordert, der ausschließlich für strukturelle Modifikationen reserviert ist.

2.2.3 Anpassung der Aufrufstellen und Resultat

Sämtliche Aufrufstellen im Kernel wurden entsprechend ihrer Zugriffsmuster refaktoriert. Modifizierende Operationen – insbesondere das Delegieren (*Sharing*) oder Entziehen (*Revocation*) von Rechten sowie das Registrieren von Naming-Capabilities – fordern den Lock nun explizit via `cspace.invoke_mut()` an. Standardmäßige Abfragen nutzen den parallelen `invoke()`-Pfad.

Hierdurch entfällt der gegenseitige Ausschluss bei rein lesenden Zugriffen im Dispatcher vollständig, während die Datenintegrität bei Modifikationen gewahrt bleibt.

2.3 Behebung von Kernel-Panics im Fehlerpfad des Dispatchers

2.3.1 Fehleranalyse

Wird ein Systemaufruf über den CSpace abgewickelt, muss der Kernel fehlgeschlagene Autorisierungen (z. B. fehlende Rechte, ungültige Deskriptoren oder ein abgewiesener Lock via `try_read`) abfangen und kontrolliert an den User-Space melden.

Im Stand der Vorarbeit führte ein Validierungsfehler jedoch reproduzierbar zu einem Kernel-Absturz (*Panic*). Die Ursache lag in der Resolver-Funktion `get_capability_entry()`:

```
1 // Fehlerhafter Aufruf:  
2 permission_denied() as *const ()
```

Statt der Funktionsadresse der Fehlerbehandlungsroutine wurde der Rückgabewert des Funktionsaufrufs `permission_denied()` als Rohzeiger gecastet. Der Low-Level-Dispatcher interpretierte diesen Rückgabewert fälschlicherweise als Zieladresse für den Funktionssprung, sprang in ungültigen Speicher und löste damit eine Kernel-Panic aus.

2.3.2 Korrektur und Stabilität

Die Behebung erfolgte durch die Übergabe des tatsächlichen Funktionszeigers:

```
1 // Korrigierter Funktionszeiger:  
2 permission_denied as *const ()
```

Der Dispatcher verzweigt im Fehlerfall nun zuverlässig in die vorgesehene Routine, belegt das Register `rax` mit dem entsprechenden Fehlercode (z. B. `-EPERM`) und kehrt geordnet in den User-Space zurück.

2.4 Rebase und Synchronisation mit dem aktuellen Kernel-Stand

Der Capability-Branch basierte auf dem Stand von September 2025. Um die Weiterentwicklung (insbesondere Shared Memory und Messaging) voranzutreiben, war ein vollständiger Rebase auf den aktuellen Entwicklungszweig des Betriebssystems erforderlich.

2.4.1 Konsolidierung der ABI und Datenstrukturen (`shared_types.rs`)

Zur Vereinheitlichung der Schnittstelle zwischen Kernel und User-Space mussten zentrale Datenstrukturen angepasst werden:

- **Rechte- und Dateioperationen:** Die `struct Capability` wurde in `shared_types.rs` überführt. Um das Teilen von Capabilities über reguläre Dateioperationen zu unterstützen, wurde das Flag `SHARE` als neue `OpenOption` ergänzt.
- **ABI-Verschiebungen:** Durch parallele Entwicklungen im Hauptzweig hatten sich fundamentale Enums verschoben; diese Diskrepanzen wurden synchronisiert (u. a. `SeekOrigin::Current` von 3 auf 4 sowie `FileType::Link` von 10 auf 16), um Fehlinterpretationen auf Systemaufrufebene zu verhindern.

2.4.2 Namensraum-Restriktionen und Sicherheitsaspekte (`naming/lib.rs`, `sys_naming`)

Ein struktureller Konflikt zwischen dem traditionellen hierarchischen Dateisystem und dem strikten Capability-Paradigma trat im Naming-Subsystem zutage:

- **Isolation und Zugriffsbeschränkung:** Im vorliegenden Capability-Modell besitzt ein Prozess keinen impliziten Zugriff auf den gesamten Verzeichnisbaum, sondern ausschließlich auf jene Ordner und Ressourcen, für die ihm explizit ein Token delegiert wurde.
- **Deaktivierung von `sys_readdir` und `cwd`:** Funktionen wie `sys_readdir` sowie das Abfragen bzw. Setzen des aktuellen Arbeitsverzeichnisses (`cwd` in `namimg/lib.rs`) setzen voraus, dass Verzeichnishierarchien global traversiert werden können. Würde einem Prozess das unbeschränkte Auslesen fremder Ordnerinhalte gestattet, liefe dies dem Schutzkonzept zuwider. Da sich dieses Problem aufgrund der fundamentalen Struktur des Capability-Systems nicht trivial auflösen ließ, wurden `sys_readdir` und `cwd` vorerst deaktiviert bzw. auskommentiert.

Compilerwarnungen in `namimg/lib.rs` wurden im Rahmen dieses Schritts bewusst zurückgestellt, um die Portierung nicht durch rein kosmetische Änderungen zu verpflichten.

2.4.3 Überarbeitung des Syscall-Dispatchers und Low-Level-Register-Management

Die weitreichendsten Änderungen erfolgten im Assembly-Dispatcher des Kernels:

- **Ablösung der Syscall-Table:** Die ehemals statische Systemaufruf-Tabelle des Hauptsystems wurde entfernt. Stattdessen wird jeder Systemaufruf dynamisch über `get_capability_entry()` aufgelöst, womit die Autorisierung fest in den Dispatching-Pfad eingebettet ist.
- **Registersicherung für `get_capability_entry`:** Da der Aufruf von `get_capability_entry` Register überschreiben würde, müssen die CPU-Register an dieser Stelle im Dispatcher erneut gesichert werden (`// Save all Registers again because get_capability_entry would overwrite them`).
- **Entfernen von `r12-r15` und Register-Begradigung:** Die Callee-Saved-Register `r12`, `r13`, `r14` und `r15` wurden im Zuge der Systemweiterentwicklung an dieser Stelle nicht mehr benötigt. Um den Stack-Overhead während der zusätzlichen Sicherung zu minimieren und den Mehraufwand zu kompensieren, wurden `push-` und `pop-` Operationen für `r12-r15` vollständig aus dem Dispatcher entfernt.
- **Korrektur der Stack-Reihenfolge:** Die Wiederherstellung von `rbx` wurde korrigiert und logisch hinter `rcx` eingereiht, um eine korrekte Stack-Ausrichtung zu gewährleisten und Fehlbelegungen bei der Rückkehr aus dem Trap-Handler auszuschließen.

2.4.4 Testumgebung und Verifikation (`api.rs`, `test/pipe-rs`)

Um die korrekte Funktion des zusammengeführten Kernels zu verifizieren, wurde die Test-Suite isoliert:

- In `api.rs` wurden gezielte Funktionstests für die Capability-Schnittstellen integriert.

- In `test/pipe-rs` wurden neuere Testfälle (`test1` und `test2`) temporär aus der Testausführung entfernt. Auf diese Weise konnte das Zusammenspiel aus Interprozesskommunikation (Pipes) und CSpaces deterministisch und frei von Nebeneffekten anderer, noch in Entwicklung befindlicher Subsysteme validiert werden.

Kapitel 3

Gliederung der Arbeit

Eine sinnvolle und korrekte Unterteilung der Arbeit ist nicht nur wichtig für den Leser, sondern hilft auch dem Verfasser bei der Anfertigung.

Allgemeiner grundlegender Aufbau mit den wichtigsten inhaltlichen Aspekten:

1. Einleitung: Hinführung an die Thematik, Problemstellung, Ziel der Arbeit, Aufbau der Arbeit
2. Grundlagen: Erläuterung von Begriffen, Definitionen, Algorithmen, Programmier-techniken, etc. die im Kontext der Arbeit Verwendung finden und welche der Leser für das Verständnis benötigt. Hierbei muss nicht ein ganzer Themenbereich der Informatik ausgearbeitet und erklärt werden. Referenzen auf Literatur sind hier sehr wichtig.
3. Hauptteil: Der Hauptteil der Arbeit besitzt keine vorgegebene Gliederung. Hier muss eine dem Thema angemessene Unterteilung in ein oder mehrere Kapitel und Unterkapitel ausgearbeitet werden. Oft bietet sich hier eine Trennung von Design/Architektur und Implementierung an.
4. Evaluation: Messungen von selbstgeschriebenen oder bereits vorhanden Benchmarks müssen in diesem Abschnitt mithilfe von Tabellen und Grafiken dokumentiert werden. Weiterhin muss der Versuchsaufbau festgehalten werden. Dabei sind alle in den Messungen verwendeten Systeme mit ihren (relevanten) Hardware- und Softwarekonfiguration aufzuführen. Die Ergebnisse müssen bewertet und ausführlich erklärt werden. Ausreisser oder Anomalien sind nicht außer Acht zu lassen!
5. Zusammenfassung, Fazit und Ausblick: Zum Schluss werden die vorgestellten Konzepte und die Implementierung zusammengefasst. Ein Bezug auf die erfüllten/nicht erfüllten Ziele der Arbeit sowie die Ergebnisse der Evaluation dürfen hier nicht fehlen. Weiterführende Ideen und Konzepte, die über die Themenstellung hinausgehen und Verbesserungspotential sind mögliche Punkte für einen Ausblick.

Kapitel 4

Struktur

Die Arbeit kann mit den nachfolgenden Befehlen sehr einfach in Kapitel mit Unterkapiteln eingeteilt werden.

- `\chapter{Name}` - Beginne ein neues Kapitel
- `\section{Name}` - Beginne einen neuen Abschnitt eines Kapitels
- `\subsection{Name}` - Beginne einen Unterabschnitt eines Abschnitts
- `\subsubsection{Name}` - Beginne einen Unterabschnitt in einem Unterabschnitt

Mithilfe von Labels lassen sich Referenzen erstellen.

- `\label{LabelName}` - Erstelle ein Label
- `\ref{LabelName}` - Referenziere ein Label

4.1 Mein Abschnitt

4.1.1 Unterabschnitt

In diesem Abschnit wird beschrieben...

4.1.2 Noch ein Unterabschnitt

Noch tiefere Hierarchie

Im Unterabschnitt 4.1.1 wurde erläutert...

Noch ein Unterpunkt vom Unterpunkt

4.1.3 Wieder eine Subsection

4.2 Und eine einfache Section

4.3 Zeilenumbrüche und Absätze

Manchmal ist es ganz gut (hauptsächlich am Ende der Arbeit), die Strukturierung etwas in die Hand zu nehmen, da LaTeX nicht immer eine optimale Aufteilung des Textes mit Grafiken und Tabellen erzeugt. Ein Zeilenumbruch lässt sich mit `\` erzeugen.

Falls ein Umbruch auf einen weiteren folgt, so erzeugt dies einen Absatz.

4.4 Seitenumbrüche

Auch das erzwingen einer neuen Seite kann nützlich sein, um eine bessere Formatierung zu bekommen. Mit `\newpage` lässt sich dies erzwingen.

Kapitel 5

Organisation

Folgende Makros können während des Schreibens der Ausarbeitung sehr nützlich sein.

1. `\todo{Text}`

Auffälliges Markieren für TODOs

2. `\begin{verbatim}Text\_end{verbatim}\end{verbatim}`

Text in diesem Block wird von Latex nicht ausgewertet (Sonderzeichen, Latex-Kommandos)

Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet clita kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit amet. Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet clita kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit amet.

Referenz
hier ein-
fügen

Inline todo



Übersicht Architektur

Kapitel 6

Text

Einige allgemeine Punkte zum Schreiben von Text:

- *kursiv*
- **fett**
- „deutsche Anführungszeichen“
- Umlaute ä, ö, ü und ß
- “englische Anführungszeichen”
- Wörter_durch_Unterstriche_getrennt
- Schreibmaschinenschrift
- `Variable`

Kapitel 7

Zitieren

Referenzen aus dem Literaturverzeichnis immer mit

```
\cite{CiteKey}
```

kenntlich machen. Der **CiteKey** wird bei jedem BibTex-Eintrag festgelegt, z.B.: [1]
Falls dieser Key fehlt, so wird anstatt einer Zahl in eckigen Klammern ein [?] angezeigt.
Nicht verwendete/referenzierte Literatur wird automatisch nicht im Literaturverzeichnis
aufgeführt.

Kapitel 8

Auflistungen

8.1 Enumeration

1. Erster wichtiger Punkt der Liste
2. Ein weiterer
3. Noch einer

8.2 Bullet Points

- Apfel
- Birne
- Gurke

8.3 Eigene Symbole

- Apfel
- Birne
- Gurke

Kapitel 9

Bilder und Grafiken

Bilder bzw. Grafiken lassen sich auf verschiedene Arten mit LaTeX einbinden. Es gilt zu beachten, dass im Text ein inhaltlicher Bezug mithilfe von

```
\ref{label_name_grafik}
```

hergestellt ist.

Beispiel: In Darstellung 9.1 ist das Logo der HHU zu sehen, welches ...

Viele Dateiformate werden unterstützt, jedoch ist die Nutzung von jpg, png, gif oder pdf (Vektorgrafiken) zu empfehlen.



Abbildung 9.1: Einzelne Grafik (centered)



Abbildung 9.2: Einzelne Grafik (verkleinert)



(a) HHU Logo links

(b) HHU Logo rechts

Abbildung 9.3: Zwei Grafiken horizontal angeordnet



(a) HHU Logo oben



(b) HHU Logo unten

Abbildung 9.4: Zwei Grafiken vertikal angeordnet

Kapitel 10

Tabellen

Tabellen sind vor allem im Abschnitt der Evaluation sehr wichtig. Die Messergebnisse aus der Tabelle müssen im Text erklärt und referenziert werden.

10.1 Einfache Tabelle

Anzahl der Iterationen	100	250	500	1.000
Implementierung 1 (ms)	0,375738	0,51265	0,77477	1,201552
Implementierung 2 (ms)	0,397062	0,397987	0,411314	0,415611

Tabelle 10.1: Durchschnittliche Zeit pro Frame aus 10.000 Frames in Millisekunden

Die Ergebnisse aus Tabelle 10.1 zeigen, dass...

10.2 Komplexere Tabelle: Multicolumn und Multirow

<i>Datensatz1</i>	<i>Datensatz2</i>		<i>Typ</i>	
	$t_{total}(ms)$	#Pakete	$t_{total}(ms)$	#Pakete
0	45,654	2	113,692	8
1	60,385	4	138,214	16
2	78,785	6	247,939	24
3	121,979	11	389,466	44
4	241,632	20	653,795	80
5	412,281	39	1.088,248	156

Tabelle 10.2: Gesamtzeiten der Übertragungen der Datensätze 1 und 2 in Millisekunden der Typen 0 bis 5

Die Messergebnisse der Testreihe sind in Tabelle 10.2 festgehalten und zeigen...

Kapitel 11

Matheumgebung

11.1 Superscript und Subscript

- $Matrix_{4x4}$
- x^{a+b}

11.2 Zentrierte Formeln

$$x^n + y^n = z^n$$

11.3 Symbole

- $\alpha \beta \gamma \rho \sigma \delta \epsilon$
- $\times \otimes \oplus \cup \cap$
- $< > \subset \supset \subseteq \supseteq$
- $\int \oint \Sigma \Pi$

Kapitel 12

Code

12.1 Pseudocode

```
Input:  $A[0..N-1]$ , value  
Output: value if found, otherwise not_found  
// Lokale Variablen  
let  $low = 0$  let  $high = N - 1$   
// Es gibt auch noch weitere Loops wie ForEach  
while  $low \leq high$  do  
    // Invariante:  $value > A[i]$  for all  $i < low$   
    // Invariante:  $value < A[i]$  for all  $i > high$   
     $mid = (low + high) / 2$ ;  
    if  $A[mid] > value$  then  
        |  $high = mid - 1$ ;  
    else  
        | if  $A[mid] < value$  then  
            |  $low = mid + 1$ ;  
        | else  
            | return mid;  
        | end  
    end  
    return not_found;  
end
```

Algorithmus 1: Binärsuche in Pseudocode

12.2 Codeschnipsel

HelloWorld.java

Java

```

1 public class HelloWorld {
2     public static void main(String[] args) {
3         System.out.println("Hello World!");
4     }
5 }

```

HelloWorld.kt

Kotlin

```

1 fun main() {
2     println("Hello world!")
3 }

```

HelloWorld.scala

Scala

```

1 println("Hello world!")

```

hello.c

C

```

1 #include <stdio.h>
2
3 int main(int argc, char** argv) {
4     printf("Hello World!\n");
5     return 0;
6 }

```

hello.cpp

C++

```

1 #include <iostream>
2
3 int main () {
4     std::cout << "Hello world!" << std::endl;
5     return 0;
6 }

```

hello.rs

Rust

```

1 fn main() {
2     println!("Hello world!");
3 }

```

hello.go

Go

```

1 package main
2
3 import "fmt"
4
5 func main() { fmt.Println("Hello world!") }

```

12.3 Terminalausgabe

```

● ● ●
[user@computer thesis]$ ls -la
total 12
drwxr-xr-x  3 user user 4096 Apr 28 19:09 .
drwxr-xr-x 23 user user 4096 Apr 28 19:09 ..
drwxr-xr-x  7 user user 4096 Apr 28 20:12 folder

```

Literatur

- [1] M. Welsh, D. Culler und E. Brewer, “SEDA: An Architecture for Well-conditioned, Scalable Internet Services”, in *Proceedings of the Eighteenth ACM Symposium on Operating Systems Principles*, Ser. SOSP '01, Banff, Alberta, Canada: ACM, 2001, S. 230–243, ISBN: 1-58113-389-8. DOI: 10.1145/502034.502057. Adresse: <http://doi.acm.org/10.1145/502034.502057>.

Abbildungsverzeichnis

9.1	Einzelne Grafik (centered)	15
9.2	Einzelne Grafik (verkleinert)	16
9.3	Zwei Grafiken horizontal angeordnet	16
9.4	Zwei Grafiken vertikal angeordnet	16

Tabellenverzeichnis

10.1	Durchschnittliche Zeit pro Frame aus 10.000 Frames in Millisekunden . . .	17
10.2	Gesamtzeiten der Übertragungen der Datensätze 1 und 2 in Millisekunden der Typen 0 bis 5	17

Algorithmenverzeichnis

1	Binärsuche in Pseudocode	19
---	------------------------------------	----

Ehrenwörtliche Erklärung

Hiermit versichere ich, die vorliegende Projektarbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt zu haben. Alle Stellen, die aus den Quellen entnommen wurden, sind als solche kenntlich gemacht worden. Diese Arbeit hat in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen.

Düsseldorf, 7.September 2026

Marek Westermann